PX4 无人机自主飞行与自动化测试平台

最终执行方案｜学习路线、实操任务、交付物与电脑端 ChatGPT 协作说明

英文项目名：PX4-Based UAV Autonomous Flight and Automated Testing Platform

项目定位：以可复现的仿真自主飞行为基础，逐步建立遥测采集、任务执行、自动化测试、数据分析和技术文档能力。目标是做出真实、可展示、可解释的工程作品，而不是只完成软件安装。

使用方式：按阶段执行，每次只推进一个小任务。执行后把终端输出、日志、代码或截图发给 ChatGPT，由其核验结果并给出下一步。计划时间只是估算，必须以验收结果为准。

一、最终目标与项目边界

1. 最终交付物

- 可复现的 PX4 SITL 仿真环境说明：记录实际软件版本、启动方式、通信配置和已知限制。
- 系统架构图：说明 Gazebo、PX4 SITL、MAVLink、QGroundControl、MAVSDK-Python 的角色与数据流。
- Python 遥测采集程序：连接检查、位置/高度/速度/姿态读取、时间戳、CSV 保存、异常处理和数据质量检查。
- 仿真自主飞行程序：安全检查通过后完成起飞、定高悬停、降落；基础稳定后扩展航点任务。
- 可重复执行的测试用例：前置条件、操作、预期结果、实际结果、通过/失败和日志路径。
- 真实数据与分析脚本：按实际任务计算完成情况、持续时间、位置/高度误差等适用指标并绘图。
- 项目 README、技术报告、演示视频、简历项目描述和英文项目摘要。所有成果必须来自实际完成的工作。

2. 系统角色与关键区别

Gazebo 负责仿真世界、模型、物理与传感器模拟；PX4 SITL 运行飞控软件；MAVLink 负责飞控与外部系统之间的消息通信；QGroundControl 用于地面站监控与任务交互；MAVSDK-Python 用于 Python 程序访问飞控功能和遥测。具体端口、消息流与启动方式以本机实际配置和日志为准。

重要：MAVSDK 能连接或读取某个数值，不等于 Gazebo 物理仿真、传感器输入、飞控状态估计和控制闭环全部正常。项目要分别验证这些层次。

3. 当前环境基线（以用户最近一次自报为准，之后需留档核验）

- VirtualBox 中 Ubuntu 22.04 已运行；PX4 SITL 与 Gazebo 据报已启动。
- Windows 端 QGroundControl 据报曾成功连接；Ubuntu 中 MAVSDK-Python 据报可连接并读取位置。
- numpy、pandas、matplotlib、scipy 已安装。
- 本计划暂不处理当前运行故障。不要重复安装已经正常的组件，也不要因为单次报错就重装整套环境；先收集证据，再做最小改动。

二、教学与协作规则

- 一次只执行一个小任务：先说明目的，再给命令/代码，说明执行位置、预期结果和失败时应提供的证据。
- 明确区分 Windows PowerShell、Ubuntu 普通终端、PX4 启动终端、Python 文件和 QGroundControl 界面。
- 每个任务必须有验收标准；没通过就停在当前任务，不越级。
- 安装、删文件、改配置、重编译、修改网络或启动多个进程前，先解释风险和回滚办法。
- 所有性能数字、测试次数、误差和成功率都必须由真实日志计算；禁止编造结果。
- 优先使用仿真。涉及解锁、起飞、Offboard 或控制模式时，先核实仿真状态、预检条件、超时处理和安全退出。
- 每次任务结束记录：完成内容、证据、问题、知识点、Git 提交（如适用）和下一项任务。

三、分阶段路线图

建议以约 4 周为初始节奏；实际可能更长。阶段切换由验收标准决定，不由日期决定。

阶段	预计投入	核心结果
0. 环境与架构	1 – 2 天	确认仿真链路、版本、数据流，画架构图
1. 遥测采集	3 – 5 天	Python 采集脚本与真实 CSV 数据
2. 自主飞行	4 – 6 天	起飞、悬停、降落，之后做航点任务
3. 自动化测试	4 – 7 天	测试用例、自动判定、可复现日志
4. 数据分析	3 – 5 天	指标计算、图表、结论与局限
5. 项目包装	3 – 5 天	仓库、README、报告、视频、简历与面试

阶段 0：环境确认与系统架构（1 – 2 天）

阶段目标：建立可信起点，理解组件分工和数据流；不是重装环境，而是确认当前状态。

执行任务

- 记录 Ubuntu、PX4、Gazebo、QGC、MAVSDK-Python、Python 实际版本，写入 docs/environment.md。
- 检查进程与启动终端日志，确认 PX4 与 Gazebo 是否运行，模型是否随物理仿真运动。
- 分别验证仿真传感器输入、飞控状态、QGC 遥测和 MAVSDK 数据是否持续更新。
- 画出组件数据流图，并标注运行系统、通信方式与已确认/未确认的链路。
- 发现异常时记录现象、日志、最近改动和复现步骤；先诊断，不同时改多个变量。

阶段交付物

- docs/environment.md、架构图、docs/known_issues.md。

验收标准

- 能解释组件分工与 SITL/HITL 区别；有证据证明数据持续更新；能区分进程存在、通信连接、数据更新与闭环正常。

必须能讲清楚的问题

- PX4、SITL、HITL、Gazebo、QGC、MAVLink、MAVSDK 分别是什么？为什么 MAVSDK 连通不代表仿真闭环正常？

阶段 1：Python 遥测采集与数据记录（3 – 5 天）

阶段目标：把 “ 能连接并读到位置 ” 升级成稳定、可复现、能留存数据的程序。

执行任务

- 在 Ubuntu 用户目录创建 px4-uav-validation/scripts、logs、analysis、docs。
- 先写连接检查：等待连接状态、设置超时与异常处理，输出明确结果。
- 逐步订阅位置/高度、速度、姿态、飞行模式和解锁状态；每加一类数据先验证字段与更新频率。
- 为每条样本添加时间戳，写入 CSV；每次运行使用不同文件名，避免覆盖原始数据。
- 检查空文件、缺失字段、时间戳重复/倒退、采样间隔异常和明显异常值。
- 学习 Python 函数、模块、异常处理、async/await 与 asyncio。

阶段交付物

- scripts/telemetry_logger.py、实际采集的 CSV、docs/data_dictionary.md。

验收标准

- 脚本可独立运行；失败原因明确；CSV 有表头、时间戳和真实数据；能解释字段单位与相对/绝对高度。

必须能讲清楚的问题

- 为什么使用异步遥测订阅？怎样发现断流、重复时间戳和错误单位？

阶段 2：自主飞行任务（4 – 6 天）

阶段目标：在仿真中实现有状态检查、有超时、有结果验证的基本自主飞行。

执行任务

- 核对当前 PX4/MAVSDK 版本支持的 API；不要混淆 Action、Mission 和 Offboard。
- 实现飞行前检查：连接、飞控状态、位置/高度有效性、模式和解锁前置条件。
- 按最小步骤实现起飞、验证高度变化、悬停、降落并验证着陆状态。
- 所有等待设置超时，捕获异常；失败时给出明确原因并按适用情形安全结束。
- 基本任务稳定后再增加航点任务，为每个航点定义到达判据、等待上限和失败处理。
- 记录指令时间、飞控状态、实际遥测、任务结果和日志路径。

阶段交付物

- scripts/takeoff_hover_land.py、真实遥测日志、简短演示视频；稳定后再增加 waypoint_mission.py。

验收标准

- 多次运行并保留真实日志；程序不仅发指令，还能验证实际结果；能解释 Action、Mission、Offboard 的差异。

必须能讲清楚的问题

- 为什么命令被接受不等于任务完成？如何设计超时、异常处理和安全退出？

阶段 3：自动化测试与可控变量实验（4 – 7 天）

阶段目标：从手动演示进阶到可重复运行、自动判定并保存结果。

执行任务

- 设计连接、遥测、起飞、悬停、降落、航点、日志完整性和错误处理用例。
- 每条用例记录编号、目的、前置条件、输入参数、步骤、预期/实际结果、判定规则与日志位置。
- 将任务执行和结果判定分开，统一输出 PASS/FAIL/ERROR，并保存失败原因。
- 先跑基线测试，固定软件版本、初始条件、任务参数和环境配置。
- 基线稳定后再研究当前版本支持的风场或传感器噪声；先核对版本与插件配置。
- 一次只改变一个因素，保留配置快照；先少量验证，再决定是否做更多重复或 Monte Carlo 实验。

阶段交付物

- 测试用例表、scripts/run_test_suite.py、独立测试日志、docs/experiment_design.md。

验收标准

- 可从统一入口执行至少一组真实测试；失败不会被误判为通过；变量控制和实验配置可追溯。

必须能讲清楚的问题

- 自变量/因变量/控制变量是什么？基线测试有什么用？Monte Carlo 能证明什么、不能证明什么？

阶段 4：数据分析与技术报告（3 – 5 天）

阶段目标：使用真实 CSV 生成可解释的指标、图表和结论。

执行任务

- 用 Pandas 读取数据，检查缺失值、时间戳连续性和异常值；原始文件只读保留。
- 按任务选择指标：完成率、持续时间、悬停高度误差、航点误差、均值、标准差、最大值和 95 百分位。
- 计算水平位置误差时不能直接把经纬度角度当米；使用合适的坐标转换或测地距离。
- 绘制高度—时间曲线、误差曲线和条件对比图；只有数据支持时才使用箱线图或 CDF。
- 结论要说明样本量、条件、异常、不确定性和局限，避免夸大因果关系。

阶段交付物

- analysis/analyze_logs.py、summary.csv、图表、docs/technical_report.md 或 PDF。

验收标准

- 分析可复现；图表有单位、标题和数据来源；结论可追溯到日志和代码。

必须能讲清楚的问题

- 均值、标准差和 95 百分位分别说明什么？样本量较小时结论有何限制？

阶段 5：工程化交付、简历与面试（3 – 5 天）

阶段目标：整理成别人能运行、检查和理解的工程作品。

执行任务

- 整理 Git 仓库、依赖说明、配置样例和文档；不要提交虚拟机镜像、大型构建产物或隐私凭据。
- README 写清目标、架构、环境版本、启动步骤、输出文件、测试方法、结果和已知限制。
- 准备演示视频：环境状态、QGC、Python 任务、日志和分析图表；只展示实际完成功能。
- 撰写中文技术报告和英文项目摘要。
- 简历只写已实现且有证据的功能；计划中的功能不能写成已完成。
- 练习架构、接口、异常处理、数据质量、实验设计和结果限制相关面试题。

阶段交付物

- Git 仓库、README、架构图、测试用例、日志、分析图表、视频、英文摘要和面试笔记。

验收标准

- 按 README 可复现核心流程或明确说明依赖条件；成果与证据对应；能用约 2 分钟说明项目目标、工作、验证和局限。

必须能讲清楚的问题

- 如何证明结果可信可复现？当前项目最大局限是什么？下一步会优先改进什么？

四、建议的项目目录结构

```
~/px4-uav-validation/
■■■ README.md
■■■ requirements.txt
■■■ scripts/
■   ■■■ connection_check.py
■   ■■■ telemetry_logger.py
■   ■■■ takeoff_hover_land.py
■   ■■■ waypoint_mission.py
■   ■■■ run_test_suite.py
■■■ logs/
■   ■■■ raw/
■   ■■■ test_runs/
■■■ analysis/
■   ■■■ analyze_logs.py
■   ■■■ summary.csv
■   ■■■ figures/
■■■ docs/
    ■■■ environment.md
    ■■■ architecture.png
    ■■■ data_dictionary.md
    ■■■ test_cases.csv
    ■■■ experiment_design.md
    ■■■ technical_report.md
    ■■■ known_issues.md
```

目录按阶段逐步创建，不要求第一天就把所有文件填满。requirements.txt 只记录实际依赖，脚本名称也可以按最终实现调整。

五、每次学习的标准工作流

- 1）说明当前阶段、任务编号与上次验收结果。

- 2）ChatGPT 只安排一个最小任务，明确执行位置、命令含义、预期输出和失败时要提交的证据。
- 3）先读懂命令和风险再执行；不理解涉及删除、提权、网络或配置修改的命令时，先问清楚。
- 4）原样粘贴终端输出或上传截图/文件，不只说“ 成功了 ”。
- 5）ChatGPT 根据证据判定通过/未通过；未通过就留在当前任务。
- 6）通过后更新进度记录、保存日志与代码；需要时提交 Git，并生成下一任务提示。

六、手机与电脑 ChatGPT 同步方案

方案 A：同一账号、同一项目对话

- iPhone 与电脑浏览器/桌面应用登录同一个 ChatGPT 账号。
- 尽量在同一条项目对话中继续；若界面支持 Projects，可创建专用项目并上传本 PDF、交接提示词和项目文档。
- 手机适合阅读计划、讨论概念、拍摄/上传终端截图；电脑适合终端操作、编辑代码、管理文件和查看长日志。
- 对话同步不等于本地终端实时共享，也不自动授予电脑控制权限。

方案 B：用文件保持跨对话连续性

- 将本 PDF、交接提示词和 progress.md 放在电脑项目目录，也可放在自己的云盘。
- 每次结束时更新 progress.md：当前阶段、已验收内容、环境、证据路径、未解决问题和唯一下一任务。
- 换设备或新开对话时上传 progress.md 与必要日志，不依赖聊天记忆，也不上传大量无关文件。
- 代码用 Git 管理，原始日志按日期/任务编号保存，避免覆盖。

方案 C：电脑端控制能力的边界

电脑端 ChatGPT 可以根据你提供的终端输出、截图和文件指导操作。只有在你实际使用的客户端明确提供并授权电脑操作/代理功能时，才可能直接操作部分界面。不要假定 ChatGPT 已看见桌面、能访问 VirtualBox 或能自动执行本机命令。没有相应功能时，由你执行命令并回传结果。删除文件、提权、修改系统配置、安装软件、网络/防火墙和真实飞行控制等高影响操作，必须先解释并取得明确确认。

七、电脑端 ChatGPT 项目设置

- 项目名称：PX4 无人机自主飞行与自动化测试平台。
- 项目资料：本 PDF、Desktop_ChatGPT_Project_Handoff_Prompt.txt、最新 progress.md、环境版本记录、架构图和必要日志。
- 固定教学原则：循序渐进；一次一个任务；先解释再执行；以实际证据验收；不重复安装已正常组件；不虚构结果。
- 每次开场说明当前阶段、任务编号、当前环境、已通过成果和本次唯一目标。
- 新开对话时先粘贴交接提示词，再上传最新进度记录。

八、进度记录模板

```
■■■PX4 ■■■■■■■■■■■■■■■■
■■■■ / ■■■■■
■■■■■
■■■■■Windows / Ubuntu / PX4 ■■ / Python■■
■■■■■■■■■■■■
■■■■■■■■■■■
■■■■■■■■ / ■■■ / ■■■■
■■■■■■■■
■■■■■■■■
■■■■■■■■■
Git commit■■■■■■
■■■■■■■■■■■■■
■■ ChatGPT ■■■■■■■■■
```

九、简历表述原则

下面只是未来功能完成后的表述模板，不代表目前已经实现。最终必须根据真实代码、日志和测试证据改写。

- 搭建并验证基于 PX4 SITL、Gazebo 与 QGroundControl 的无人机仿真测试环境。
- 基于 MAVSDK-Python 开发遥测采集与 CSV 日志记录程序，并实现数据完整性检查。
- 在仿真环境中实现带状态验证、超时与异常处理的自主飞行任务。

- 使用 Pandas、NumPy 与 Matplotlib 对真实仿真日志进行指标计算和可视化分析。
- 设计可重复执行的测试用例，比较不同实验条件下的任务表现，并说明样本量与实验限制。

> 没有真实实验记录时，不得声称完成 100 次测试、达到某个成功率、抗风能力提升多少或误差降低多少。

十、电脑端教学开场语

> 请按《PX4 无人机自主飞行与自动化测试平台最终执行方案》教学。先确认我目前的进度，只给我一个最小任务。明确告诉我在哪个终端执行、命令含义、预期输出、失败时要发什么证据。先验收，再继续。